

ShareGear — MVP Functional Specification

Principal Product Owner & Senior AI Solutions Architect Review

Document Version: 1.0 — MVP Scope

Stack: Angular · ASP.NET Core (.NET 8/9) · Microsoft.SemanticKernel.Agents · SQL Server · Qdrant · SignalR

Team: Team 5 · Graduation Project

Table of Contents

1. [Executive Summary & MVP Scope](#)
 2. [User Roles & Permission Model](#)
 3. [Module 1 — On-Demand Heavy Equipment Service Booking](#)
 4. [Module 2 — Used Equipment & Spare Parts Marketplace](#)
 5. [Module 3 — Real-Time Communication & Operational Tracking](#)
 6. [Native .NET Agentic AI Integration](#)
 7. [Proactive Business Logic & Smart MVP Extensions](#)
 8. [Admin Dashboard & Operations Center](#)
 9. [Security & Compliance Framework](#)
 10. [MVP Implementation Roadmap](#)
-

PART I: Executive Summary & MVP Scope

1.1 Product Vision

ShareGear redefines the industrial services market by replacing the high-risk, low-trust model of peer-to-peer equipment rental with a **managed service delivery platform**. Instead of transferring equipment ownership or custody to a stranger, every transaction on ShareGear is structured as a **service request**: the customer specifies the task, the provider brings the equipment and the operator, and the platform orchestrates the entire lifecycle—from discovery to payment release.

The critical differentiator is **embedded AI intelligence**. Rather than static keyword search and manual admin review, ShareGear uses Microsoft Semantic Kernel to power a multi-agent backend that understands natural language, verifies listings autonomously, moderates content,

and proactively guides users toward the right service match—without replacing human judgment on final decisions.

1.2 MVP Boundary Definition

The MVP is scoped to deliver a **commercially testable, end-to-end operational platform** across the following functional pillars:

Pillar	Scope in MVP	Out of MVP
Service Booking	Full lifecycle: search → book → execute → pay → review	Multi-equipment bundled packages
Marketplace	Buy/sell used equipment & spare parts	Auction-based pricing, bulk trade deals
AI Search & RAG	Natural language search, intent extraction, result synthesis	Multi-modal (image-based) search
Multi-Agent AI	Concierge Agent, Moderator Agent	Autonomous negotiation agents
Real-Time Comms	SignalR chat, push notifications	Video/voice calls, screen sharing
Payments	Escrow-gated payment release	Installment plans, credit financing
Admin Dashboard	Moderation, analytics, risk reports	Financial reconciliation tools
Geo Features	Location-based service filtering, geo-fence alerts	Full logistics routing engine

1.3 North-Star MVP Success Metrics

- A provider can list a service and receive a verified booking in under 10 minutes.
- A customer can find a relevant service through natural language search in under 3 steps.
- The AI Moderator Agent reviews and risk-scores a new listing within 60 seconds of submission.
- Zero escrow funds are released before a service is marked operationally complete.
- Admin workload for listing approval is reduced by 60% versus full manual review.

PART II: User Roles & Permission Model

2.1 Role Hierarchy & Definitions

Role: Guest (Unauthenticated)

A guest can browse the platform to evaluate services and marketplace listings. Their experience is intentionally limited to create a clear conversion incentive toward registration.

Permitted Actions:

- Browse published service listings with full detail pages
- Browse published marketplace product listings
- Perform AI-powered semantic search (read-only results, no booking capability)
- View provider public profiles and aggregate ratings
- View pricing tiers and availability calendars (dates only, not slot-level detail)

System Restrictions:

- No booking or purchase initiation
- No access to the real-time chat system
- No access to provider contact details
- No access to the review submission form

Conversion Triggers: Any attempt to initiate a booking, send a chat message, or view operator profiles triggers a registration prompt overlay within the Angular UI.

Role: Customer (Authenticated End User)

The primary demand-side user of the platform. Customers represent construction firms, individual contractors, homeowners, or business operators who need heavy equipment services or used equipment.

Permitted Actions:

- Full service booking lifecycle (request, track, confirm completion, review)
- Marketplace product purchases
- AI-assisted search with personalized recommendations
- Real-time chat with service providers and sellers
- Payment initiation and transaction history access

- Complaint submission
- Review submission (post-service/post-purchase only)
- Manage personal profile, saved addresses, and payment methods

Business Rule — Booking Eligibility: A customer account must have a verified phone number and at least one saved address before initiating a service booking. This is enforced at the booking entry point, not at registration, to reduce registration friction.

Role: Service Provider / Seller (Authenticated Supply-Side User)

Represents companies or sole operators that own heavy equipment and offer services. A single account can act as both a service provider and a marketplace seller.

Permitted Actions:

- Create, edit, and manage service listings
- Create, edit, and manage marketplace product listings
- Upload operator profiles and equipment documentation
- Manage a real-time availability calendar
- View and respond to incoming booking requests
- Accept or reject bookings (within the defined SLA window)
- Track earnings, completed jobs, and payout history
- Communicate with customers via the chat system
- Dispute resolution submission

Business Rule — Listing Eligibility: A provider cannot publish any service or product listing until the Moderator Agent has completed a risk assessment and an admin has provided final approval. Providers are notified in real time of the review status.

Business Rule — Operator Assignment: A provider must assign at least one operator profile to a service listing before it can be made publicly available.

Role: Admin (Platform Operations)

Platform operations staff who maintain quality, resolve disputes, and monitor the AI-driven moderation pipeline.

Permitted Actions:

- Full user management (suspend, verify, role-change)
 - Final approval or rejection of flagged listings (informed by AI risk scores)
 - Complaint resolution and arbitration
 - Escrow override (exceptional circumstances only, requires dual admin confirmation)
 - Platform analytics and reporting access
 - Configuration of AI agent thresholds and blacklisted keywords
 - Manual trigger of re-moderation for any content
-

2.2 Authentication & Authorization Model

Authentication Protocol:

- JWT-based stateless authentication issued by the ASP.NET Core Identity system
- Access tokens expire after 60 minutes; refresh tokens are valid for 14 days
- Phone number verification (OTP via SMS) is required during registration to prevent throwaway account abuse
- Email verification is required for marketplace sellers and service providers before listing creation

Authorization: Role-based access control (RBAC) is enforced at the API controller and endpoint policy level. Sensitive operations (escrow release, admin overrides) additionally require claim-level validation beyond role membership.

Session State: The Angular frontend stores the access token and user role claims in memory (not localStorage) to reduce XSS exposure. On page refresh, a silent re-authentication call is made to the refresh token endpoint.

PART III: Module 1 — On-Demand Heavy Equipment Service Booking

3.1 Service Discovery & AI-Powered Search

3.1.1 The Search Entry Point

The Angular application presents a unified search bar as the primary interaction element on the homepage. This is intentionally designed to support both keyword-style and full natural language queries, lowering the barrier for non-technical users.

Supported Query Patterns:

- Keyword: "Crane Cairo"
- Descriptive: "Need an excavator for foundation digging in Nasr City"
- Conversational: "I have a job that needs heavy lifting on the 10th floor next Thursday"

All three patterns are processed identically through the AI pipeline described in Part VI. The frontend does not distinguish between them.

Secondary Discovery Mechanisms:

- Category browsing grid: Visually organized by equipment type (Cranes, Excavators, Trucks, Generators, Scaffolding)
- Featured Providers section: Curated by platform admin, surfaced for high-rating providers
- Recently Viewed: Persisted in the customer's profile session

3.1.2 Service Listing Card

Each service listing surfaced in search results or category browsing displays the following information in its card view:

- Service title and category icon
- Provider company name and verified badge (if approved)
- Base price (hourly/daily rate indicator)
- Average rating (star display) and total review count
- Operating location radius (e.g., "Serving Cairo Governorate")
- Estimated availability indicator (Available Today / Next Available: [date])
- One-line AI-generated service summary (synthesized from listing description)

3.1.3 Service Detail Page

Upon clicking a listing card, the customer lands on the Service Detail Page, which contains:

Service Information Panel:

- Full listing title and detailed description
- Equipment specifications (model, capacity, year, condition)
- Operator profile summary (name, experience years, certifications — without personal contact information)
- Available service hours and days of the week
- Geographical service zone map (visual coverage area)
- Photo gallery (equipment and operator photos uploaded by provider)

Pricing Panel:

- Hourly rate, minimum booking hours, daily rate
- Fuel/logistics surcharge indicator (if applicable)
- Pricing notes (e.g., "Overtime rates after 8 hours")

Provider Trust Panel:

- Provider aggregate rating and total completed jobs
- Active since date
- Verified document badges (Equipment Certificate, Operator License, Business Registration)
- Recent reviews (3 most recent, paginated)

Booking Initiation CTA: A prominent "Request This Service" button anchored at the top of the page triggers the booking flow.

3.2 Service Request & Booking Flow

This is the most critical user journey in the platform. It must be low-friction for the customer while systematically capturing all information required for operational service delivery.

Step 1: Booking Request Form (Angular Multi-Step Form)

The booking form is presented as a 4-step wizard within the Angular UI.

Step 1A — Job Details:

- Job title (free text, 10-100 characters)
- Job description (optional, free text up to 500 characters)
- Service type confirmation (pre-filled from the listing, read-only)
- Equipment type (pre-filled from the listing)

Step 1B — Location & Schedule:

- Service address (map picker + manual address entry with governorate/district/street fields)
- Preferred start date (calendar picker, minimum 24-hour advance booking enforced)
- Preferred start time (hour selector, within provider's listed service hours)
- Estimated job duration (hours selector with provider's minimum enforced)

Business Rule — Advance Booking Minimum: The system enforces a minimum of 24 hours between booking request submission and the requested service start time. This provides adequate operator notification and logistics time. Emergency booking requests (under 24 hours) are routed to a separate "Priority Queue" which carries a platform-defined surcharge of 20–30% and requires admin acknowledgment before confirmation.

Step 1C — Special Requirements:

- Access requirements (e.g., "Rooftop access required," "Permit needed")
- Site contact person and phone number (for operator day-of coordination)
- Any safety notes or hazardous conditions disclosure

Step 1D — Pricing Confirmation:

- System calculates the estimated total: $\text{base rate} \times \text{estimated hours} + \text{applicable surcharges}$
- Pricing breakdown displayed clearly
- Customer acknowledges the estimate is not final (actual hours billed on completion)

Step 2: Pre-Booking AI Validation (Invisible to User, Real-Time Backend)

Before the booking request is formally created in the database, the backend executes an automated multi-point validation sequence. This is described fully in Part VI (Multi-Agent Orchestration) but functionally produces the following outcomes:

- **Operator Availability Check:** The Concierge Agent queries the provider's availability schedule for conflicts on the requested date/time. If a conflict exists, the system returns a "Provider Unavailable" signal and suggests the next three available slots.
- **Geo-validation:** The requested service address is validated against the provider's declared service zone. If out-of-zone, the customer is informed and offered an out-of-zone surcharge acceptance or re-search.
- **Provider Responsiveness Score Check:** If the provider has a below-threshold response rate (calculated as described in Part VII), the customer is notified that this provider has a slow response history and is offered an equivalent alternative service proactively.

Step 3: Booking Request Submission

Upon customer confirmation on Step 1D, the following sequence executes:

1. The Angular frontend submits the booking payload to the `POST /api/bookings` endpoint.
2. The ASP.NET Core API creates a booking record in SQL Server with status: `PENDING_PROVIDER_RESPONSE`.

3. A SignalR event is emitted to the provider's connected client, triggering a real-time notification with a booking request alert.
4. Simultaneously, an SMS and push notification are dispatched to the provider.
5. A booking confirmation email is sent to the customer with the booking reference number, a summary of the request, and a note that the provider has **2 hours** to respond.
6. A countdown timer begins in the customer's booking tracker view.

Step 4: Provider Response Window

The provider sees the incoming booking request in their dashboard with full job details.

Accept Flow:

- Provider taps "Accept Booking."
- Provider must confirm which specific operator they are assigning to the job (from their registered operators list).
- Booking status transitions to: `CONFIRMED_PENDING_PAYMENT`.
- Customer is notified immediately via SignalR, push notification, and email.
- The customer's payment flow is triggered.

Reject Flow:

- Provider taps "Reject" and must select a rejection reason from a predefined list (e.g., "Unavailable on that date," "Out of service zone," "Equipment under maintenance").
- Booking status transitions to: `REJECTED`.
- Customer is notified and the platform proactively suggests 2-3 semantically similar alternative services (Concierge Agent-driven, described in Part VI).

Non-Response (SLA Breach):

- If the provider does not respond within 2 hours, the system automatically escalates. This is described fully in Section 7.4 (Smart Backup Notification Flow).
- Booking status transitions to: `PROVIDER_UNRESPONSIVE`.

Step 5: Payment Processing (Escrow Gate)

Upon provider acceptance, the customer is directed to the payment flow.

- The system displays the confirmed pricing breakdown.
- Customer selects their saved payment method or adds a new one.
- Payment is processed via the integrated payment gateway.
- **Funds are captured into escrow** — they are NOT released to the provider at this point.

- Booking status transitions to: **ACTIVE** (payment confirmed, service scheduled).
- A job confirmation is sent to both parties with the operator name, a masked contact identifier, and the scheduled time.

Business Rule — Escrow Hold: Under no circumstances are funds released to the provider before service completion confirmation. The complete escrow release workflow is specified in Section 7.1.

Step 6: Pre-Service Day Communications

T-24 Hours: Automated reminder sent to both customer and assigned operator via push notification and SMS.

T-2 Hours: Final confirmation notification sent to both parties.

T-30 Minutes: A real-time "Operator En Route" signal can be sent by the operator through the mobile interface, updating the customer's tracking view.

Step 7: Service Execution Tracking

Once the operator marks the job as "Started" (via the provider dashboard or mobile interface), the booking status transitions to: **IN_PROGRESS**.

Customer Tracking View:

- Live status indicator (En Route / On-Site / In Progress)
- Operator name and masked identifier
- Job start time (system-recorded)
- A help/issue reporting button that triggers an immediate admin notification

Provider/Operator View:

- Active job checklist
- Ability to log job milestones (optional, for complex jobs)
- Ability to flag issues (site access denied, safety hazard, scope change requested)
- End-of-job time logging

Step 8: Service Completion & Escrow Release

When the operator marks the job as "Complete":

1. Booking status transitions to: **PENDING_CUSTOMER_CONFIRMATION**.
2. Customer receives an immediate notification: "Your operator has marked the job as complete. Please confirm completion."

3. **Customer Confirmation Window:** The customer has **4 hours** to confirm or dispute the completion.

If Customer Confirms:

- Booking status transitions to: **COMPLETED**.
- Escrow release is triggered (funds move to provider's platform wallet, minus platform commission).
- The review prompt is immediately surfaced to the customer.

If Customer Does Not Respond in 4 Hours:

- System automatically confirms completion (auto-confirmation rule) and releases escrow.
- Customer can still submit a review for 72 hours post-completion.

If Customer Disputes Completion:

- Booking status transitions to: **DISPUTED**.
 - Customer must describe the dispute reason (text + optional photo upload).
 - Admin is notified immediately.
 - Escrow is held until dispute resolution.
 - Full dispute flow described in Section 7.7.
-

3.3 Booking State Machine

The following defines all permissible booking status transitions:

DRAFT

```
└─> PENDING_PROVIDER_RESPONSE (on customer submission + payment initiation)
    │
    │   └─> CONFIRMED_PENDING_PAYMENT (on provider accept)
    │       │
    │       │   └─> ACTIVE (on payment capture success)
    │       │       │
    │       │       │   └─> IN_PROGRESS (on operator marks job started)
    │       │       │       │
    │       │       │       │   └─> PENDING_CUSTOMER_CONFIRMATION (on operator marks
complete)
    │       │       │       │       │
    │       │       │       │       │   └─> COMPLETED (on customer confirm /
auto-confirm)
    │       │       │       │       │       │
    │       │       │       │       │       │   └─> DISPUTED (on customer dispute
submission)
    │       │       │       │       │       │       │
    │       │       │       │       │       │       │   └─> RESOLVED_RELEASED (dispute resolved,
funds released)
    │       │       │       │       │       │       │       │
    │       │       │       │       │       │       │       │   └─> RESOLVED_REFUNDED (dispute resolved,
customer refunded)
    │       │       │       │       │       │       │       │       │
    │       │       │       │       │       │       │       │       │   └─> REJECTED (on provider reject)
    │       │       │       │       │       │       │       │       │       │
    │       │       │       │       │       │       │       │       │       │   └─> PROVIDER_UNRESPONSIVE (on SLA breach, 2hr no response)
    │       │       │       │       │       │       │       │       │       │       │
    │       │       │       │       │       │       │       │       │       │       │   └─> REASSIGNED (on successful backup provider match)
    │       │       │       │       │       │       │       │       │       │       │       │
    │       │       │       │       │       │       │       │       │       │       │       │   └─> CANCELLED_REFUNDED (if no backup found within 4hr)
```

3.4 Service Listing Management (Provider Side)

Provider Creates a Service Listing

Step 1 — Basic Information:

- Service title
- Service category selection (predefined taxonomy: Cranes, Excavators, Bulldozers, Trucks/Moving, Generators, Scaffolding, Other)
- Detailed service description (minimum 100 characters, AI grammar and completeness check on submission)
- Service tags (auto-suggested by AI based on description content)

Step 2 — Equipment Details:

- Equipment type and model
- Equipment capacity/specifications
- Equipment condition (Excellent / Good / Fair)
- Year of manufacture
- Equipment registration number (for verification)

- Photo upload (minimum 3 photos required, maximum 10)

Step 3 — Operator Profiles:

- Provider must register at least one operator before creating a service.
- Operator fields: Full name, years of experience, specialization, license type, license number, license expiry date, profile photo.
- Operator documents uploaded here (license scans) and queued for Moderator Agent OCR verification.

Step 4 — Pricing & Availability:

- Hourly rate and/or daily rate
- Minimum booking duration (hours)
- Operational hours (by day of week)
- Service zone: Governorate selection + radius definition on map
- Blackout dates (maintenance windows, holidays)

Step 5 — Document Verification:

- Equipment ownership certificate or lease agreement
- Business registration certificate (for company providers)
- Insurance certificate (encouraged, displays verified badge)
- All documents submitted to Moderator Agent OCR pipeline

Step 6 — Submission:

- Listing enters `PENDING_REVIEW` status.
- AI Moderator Agent processes within 60 seconds.
- Admin receives risk report and makes final approval decision.
- Provider notified of approval or rejection with specific reasons.

PART IV: Module 2 — Used Equipment & Spare Parts Marketplace

4.1 Marketplace Architecture & Scope

The Marketplace module operates as a distinct sub-platform within ShareGear, sharing authentication, moderation, and payment infrastructure but with its own listing type, product model, and purchase flow. It is a **fixed-price, buy-now** marketplace for MVP scope.

- Used heavy equipment (excavators, cranes, generators — large items)
 - Used power tools (drills, saws, grinders — medium items)
 - Workshop & carpentry equipment
 - Spare parts (mechanical, hydraulic, electrical)
 - Scaffolding materials
 - Construction consumables
-

4.2 Seller Listing Creation Flow

Step 1: Product Information

- Product title
- Category (hierarchical two-level: Category → Sub-category)
- Condition: New / Like New / Good / Fair / For Parts Only
- Year of manufacture (for equipment)
- Detailed description (minimum 80 characters)
- Key specifications (structured fields vary by category — e.g., for a generator: KVA rating, fuel type, phase type)

Step 2: Pricing

- Listing price (fixed)
- Negotiable toggle (Yes/No — if Yes, the buyer can submit an offer through the chat system)
- Location of item (governorate and district — for buyer proximity awareness)
- Seller's preferred transaction method: Buyer collects / Seller delivers (within radius) / Shipping only

Step 3: Media Upload

- Minimum 3 product photos required
- Maximum 12 photos
- Optional: Video upload (max 60 seconds)
- AI Moderator Agent performs basic image content check on upload

Step 4: Document Upload (Optional but Incentivized)

- Equipment certificate or purchase receipt → triggers "Verified Ownership" badge

- Maintenance records → triggers "Service History Available" badge
- These badges are displayed prominently on the listing and increase buyer trust

Step 5: Moderation & Publishing

- Listing enters `PENDING_REVIEW` status on submission
 - Moderator Agent processes within 60 seconds (risk scoring, price anomaly detection, content check)
 - Admin reviews AI risk report and approves or rejects
 - Approved listings move to `ACTIVE` and are immediately indexed for semantic search
-

4.3 Buyer Discovery & Purchase Flow

Discovery

Buyers can discover marketplace products through:

- Unified AI search bar (same search pipeline as services, with domain context switching)
- Category browsing with filter sidebar (price range, condition, location, seller rating)
- "Similar Items" recommendations on product detail pages (AI-powered, vector similarity)

Product Detail Page

- Full photo gallery with zoom
- Pricing and negotiation indicator
- Seller profile panel: rating, total items sold, member since, response rate
- Verified document badges
- Location indicator with approximate distance from buyer's saved address
- Product description and specifications
- Related products (AI-suggested)
- "Contact Seller" button (opens direct SignalR chat)
- "Buy Now" button (initiates purchase flow)

Purchase Flow

Step 1 — Order Confirmation:

- Buyer reviews item, price, and logistics terms
- Selects delivery preference (if seller offers options)
- Enters or confirms delivery address

Step 2 — Payment:

- Payment processed through gateway
- Funds enter escrow
- Seller is notified of sale immediately
- Buyer receives order confirmation

Step 3 — Logistics Coordination:

- For Buyer-Collects: Buyer and seller coordinate via the platform chat; seller confirms collection date/time
- For Seller-Delivers: Seller confirms delivery date and updates tracking status in dashboard
- For Shipped Items: Seller enters tracking number; buyer can view tracking status

Step 4 — Buyer Confirmation & Escrow Release:

- Buyer confirms item received as described within **72 hours** of confirmed delivery/collection
 - If buyer does not confirm within 72 hours: auto-confirmation rule triggers and escrow releases
 - If buyer disputes: **DISPUTED** status, admin intervention (described in Section 7.7)
-

4.4 Marketplace Product State Machine

```
DRAFT
  ↳ PENDING_REVIEW      (on submission)
    ↳ ACTIVE             (on admin approval)
      ↳ SOLD             (on purchase completion + escrow release)
        ↳ REMOVED        (seller deletes / admin removes)
          ↳ REJECTED     (on admin rejection - seller can revise and
resubmit)
```

4.5 Offer Negotiation Flow (Negotiable Listings)

For listings marked as Negotiable:

1. Buyer submits an offer amount through a dedicated offer form (not free-text chat)
2. The offer is logged in the system with a 24-hour seller response window

3. Seller can: Accept (triggers purchase flow at the offered price) / Counter (proposes a new price) / Decline
 4. Maximum 3 offer rounds before the buyer must proceed at list price or walk away
 5. All offer communications are captured in the platform's structured offer trail (not the chat), for dispute reference
-

PART V: Module 3 — Real-Time Communication & Operational Tracking

5.1 SignalR Chat System

Architecture

The SignalR hub is hosted within the ASP.NET Core application and manages persistent WebSocket connections for authenticated users. Connection state (connection ID to user ID mapping) is stored in a distributed in-memory store (Redis for production scale; in-memory for MVP) to support multi-instance deployment.

Conversation Types

Customer ↔ Provider/Seller Chat:

- Initiated by customer from the service listing or marketplace product page
- Context-linked: The conversation is automatically tagged to the relevant service or product listing
- Both parties see the listing reference at the top of the chat thread
- A customer cannot initiate a new chat thread with the same provider for the same listing if one is already active

Customer ↔ Admin Chat (Support):

- Available through the Help Center for escalated issues
- Admin side is handled through the Admin Dashboard chat queue

Message Types Supported

- Text messages (max 1,000 characters per message)
- Image attachments (max 5MB, image types only, content-checked on upload)
- System messages (automated, non-editable — e.g., "Booking #12345 was confirmed")
- Offer submission cards (structured card format for marketplace offer negotiation)

Chat Business Rules

- **No external contact sharing:** Messages are scanned for patterns matching phone numbers, email addresses, and external URLs. If detected, the message is blocked and a warning is issued. Persistent violations trigger a moderation flag.
 - **Retention:** Chat messages are stored in SQL Server and retained for 12 months post-conversation close for dispute reference.
 - **Message Read Receipts:** Single tick (sent), double tick (delivered), blue tick (read) — standard pattern.
 - **Conversation Locking:** Once a booking is marked COMPLETED and the review period has closed (72 hours post-completion), the associated conversation is locked (read-only). New conversations must be opened for future engagements.
-

5.2 Notification System

Notification Channels

All notifications are delivered across three channels simultaneously:

- **In-App:** Notification bell in Angular header, real-time via SignalR
- **Push Notification:** Via Firebase Cloud Messaging (FCM) to the mobile browser or installed PWA
- **SMS:** For critical operational notifications only (booking confirmed, booking provider unresponsive, payment captured, dispute opened)

Notification Event Catalog

Event	Customer	Provider	Admin
Booking request submitted	✓ (confirmation)	✓ (action required)	—
Booking accepted by provider	✓ (action: pay)	✓ (confirmation)	—
Booking rejected	✓ (alternatives suggested)	—	—
Payment captured	✓	✓	—
Provider unresponsive (SLA breach)	✓	✓ (escalation warning)	✓
Operator en route	✓	—	—

Event	Customer	Provider	Admin
Job started	✓	✓	—
Job marked complete	✓ (action: confirm)	✓	—
Dispute opened	✓	✓	✓ (action required)
Escrow released	—	✓	—
Listing approved	—	✓	—
Listing rejected	—	✓	—
New message received	✓	✓	—
Review published	—	✓	—
Complaint submitted	—	—	✓

5.3 Operational Tracking Dashboard

Customer Booking Tracker

Each active booking has a dedicated tracking page showing:

- Booking reference and status badge (visual state machine indicator)
- Assigned operator name and masked identifier
- Scheduled date/time and location summary
- A live status timeline (Confirmed → Payment Received → Operator Assigned → Operator En Route → Job Started → Job Completed)
- Document access: Downloadable job summary once completed

Provider Operations Dashboard

- Active jobs queue: Ordered by scheduled start time
- Pending booking requests with countdown timers
- Earnings summary: This week / This month / All time
- Availability calendar with bookings overlaid
- Operator assignment view: Which operator is on which job
- Response rate and acceptance rate metrics (provider-visible)

PART VI: Native .NET Agentic AI Integration

6.1 AI Architecture Overview

All AI intelligence in ShareGear is orchestrated entirely within the **ASP.NET Core backend** using **Microsoft.SemanticKernel** and the **Microsoft.SemanticKernel.Agents** framework. There is no direct AI SDK call from the Angular frontend. The Angular UI communicates only with ShareGear's own REST API endpoints and SignalR hub. This is the correct architectural boundary for security, cost control, API key protection, and audit logging.

The AI layer is composed of the following components:

Component	Technology	Role
LLM Provider	Azure OpenAI (GPT-4o) via SK	Natural language understanding, response generation
Embedding Model	text-embedding-3-small via SK	Vector generation for semantic search
Orchestration Framework	Microsoft.SemanticKernel.Agents	Agent lifecycle, tool routing, multi-agent coordination
Vector Store	Qdrant	Embedding storage, similarity search
AI Memory	Semantic Kernel Memory (VectorStore abstraction)	RAG context retrieval
OCR	Azure AI Document Intelligence	Document text extraction
Prompt Templates	SK PromptTemplateConfig	Structured, version-controlled prompts

6.2 Semantic Search & RAG Flow

Overview

The Semantic Search system allows users to enter natural language queries and receive highly

relevant service or marketplace results — even when their exact wording does not match listing text. This is achieved through embedding-based similarity search combined with LLM-synthesized response generation.

Full End-to-End Flow

Phase 1 — Query Reception & Context Determination

1. The user types a natural language query into the Angular search bar and submits it.
2. The Angular application sends a `POST /api/search/query` request with the following payload:
 - `query`: The raw user text (e.g., "I need something to lift furniture to a high floor in Heliopolis")
 - `context`: `"services"` | `"marketplace"` | `"all"` (determined by which section of the app the user is in)
 - `userId`: If authenticated (optional for guests)

Phase 2 — Intent Extraction via Semantic Kernel

3. The ASP.NET Core API receives the request and invokes the **Concierge Agent** (see Section 6.3).
4. The Concierge Agent passes the raw query through a structured SK prompt function:
 - **Prompt Goal**: Extract structured intent from the natural language query.
 - **Output Schema**:

```
{ "equipment_type": string, "task_description": string, "location": string, "urgency": string, "budget_signal": string, "context_domain": string }
```
 - **Example Extraction**: Query: "Need a crane to lift furniture to the 10th floor in Nasr City next week" →

```
{ equipment_type: "crane", task_description: "furniture lifting, high-floor, 10th floor", location: "Nasr City", urgency: "low", budget_signal: "unspecified", context_domain: "services" }
```
5. If the extracted location does not match a known Egyptian governorate or district, the agent flags it for clarification and returns a clarifying question response to the user (e.g., "Could you specify which area of Cairo you're located in?").

Phase 3 — Embedding Generation

6. The extracted structured intent object is serialized to a clean text representation optimized for embedding (structured but humanized: "Crane service for furniture lifting on a high floor in Nasr City").
7. The Semantic Kernel `ITextEmbeddingGenerationService` generates a 1536-dimension vector from this representation using the embedding model.

Phase 4 — Qdrant Vector Search

8. The generated embedding vector is sent to the Qdrant instance as a similarity search query.
9. Qdrant performs a cosine similarity search against the indexed collection for the relevant context domain (`services_collection` or `marketplace_collection`).
10. The top-K results (K=10 for MVP) are retrieved, each containing:
 - The listing ID (foreign key reference to SQL Server)
 - The similarity score (0 to 1)
 - The stored payload snapshot (title, category, location, base price)
11. Results with similarity score below a defined threshold (e.g., 0.65) are filtered out.

Phase 5 — SQL Server Hydration

12. The listing IDs from the Qdrant results are used to fetch full, current listing records from SQL Server. This is critical: Qdrant stores embedding snapshots, but the authoritative current data (current pricing, current availability, current status) lives in SQL Server.
13. Any listings with status `INACTIVE`, `SUSPENDED`, or `PENDING_REVIEW` are removed from the result set at this point.

Phase 6 — RAG Response Synthesis

14. The Concierge Agent assembles a RAG context block: the full listing data of the hydrated results is formatted as a structured context block and injected into the LLM prompt.
15. A synthesis prompt instructs the LLM to:
 - Rank the results by relevance to the user's specific intent
 - Generate a 1-sentence natural language explanation for why each top result matches
 - If the user's intent suggests a location mismatch, note it
 - If no results exceed the relevance threshold, generate a transparent "no match found" response with suggestions (broadening the search, checking a neighboring area)
16. The LLM produces a structured response containing: ranked listings, match explanations, and any clarifying suggestions.

Phase 7 — Response Return to Angular

17. The API endpoint returns:
 - `listings[]`: The ordered, hydrated listing objects
 - `explanations[]`: One-line match explanations per listing
 - `searchMetadata`: { `queryIntent`: parsed intent object, `resultCount`: int, `hasLowConfidence`: bool }

- Optional: `clarification_question` (if the agent identified intent ambiguity)

18. The Angular component renders the listing cards, with the AI-generated match explanations displayed as subtle sub-text under each card.

Qdrant Index Maintenance

Listings must be kept synchronized between SQL Server (source of truth) and Qdrant (search index).

- **On listing approval:** The listing description + key attributes are embedded and upserted into Qdrant with the listing ID as the point ID.
 - **On listing edit:** Re-embedding is triggered and the Qdrant point is updated.
 - **On listing deactivation/deletion:** The Qdrant point is deleted immediately.
 - This synchronization is handled by a background service (`IHostedService`) in the ASP.NET Core application, consuming events from an internal domain event queue.
-

6.3 Concierge Agent — Full Specification

Agent Identity

The Concierge Agent is a `ChatCompletionAgent` instantiated using

`Microsoft.SemanticKernel.Agents.ChatCompletionAgent`. It operates in the context of a user's interaction session and is responsible for all customer-facing intelligence: search assistance, recommendations, personalized suggestions, and real-time service matching.

Agent Instructions (System Prompt Directive)

The agent is configured with a durable system prompt that establishes:

- Its persona: A knowledgeable, professional industrial services advisor for the ShareGear platform
- Its domain constraints: It only discusses services and products available on the ShareGear platform; it does not give general industry advice
- Its tone: Professional, concise, Arabic-aware (capable of bilingual Egyptian Arabic/English responses)
- Its tool access: The agent has access to a curated set of SK plugin functions (described below)

Concierge Agent SK Plugin Functions (Tools)

SearchServicesPlugin:

- `SearchByIntent(intentObject)`: Executes the Qdrant similarity search and returns ranked listing IDs
- `GetListingDetails(listingId)`: Fetches full listing detail from SQL Server
- `GetProviderAvailability(providerId, requestedDate)`: Queries the provider's availability calendar
- `GetAlternativeServices(failedListingId, reasonCode)`: Returns semantically similar alternatives when a primary result is unavailable or rejected

UserContextPlugin:

- `GetUserSavedAddresses(userId)`: Retrieves saved addresses for location pre-population
- `GetUserBookingHistory(userId)`: Retrieves past bookings to inform personalized suggestions

GeographyPlugin:

- `ValidateLocationInServiceZone(locationCoords, providerId)`: Checks if a requested location falls within a provider's declared service zone

Concierge Agent Invocation Contexts

Context 1 — Search Query Processing: As described in Section 6.2, the agent processes every search query.

Context 2 — Booking Conflict Resolution: When a customer's desired date/time is unavailable, the agent is invoked to suggest alternatives: "Provider X is unavailable on Thursday, but Provider Y has very similar equipment and is available on both Wednesday and Friday. Provider Y has a 4.7 rating and has completed 82 similar jobs."

Context 3 — Post-Rejection Re-Engagement: When a provider rejects a booking, the agent is invoked within 30 seconds to generate a proactive re-engagement message to the customer with alternative suggestions, preventing customer drop-off.

Context 4 — Onboarding Guidance: New providers can invoke the Concierge Agent (in its provider-advisory mode) during listing creation to get AI-assisted suggestions for their service description, pricing guidance based on similar listings, and recommended service zones based on their stated location.

Agent Identity

The Moderator Agent is an **AgentGroupChat** participant that operates entirely in the backend without any direct customer-facing interaction. It is triggered by content submission events (new listings, new products, new reviews, new complaints) and produces structured risk assessment reports for human admin review.

The agent is implemented as a **ChatCompletionAgent** with specialized plugins, and in multi-agent scenarios, it communicates with the Concierge Agent through a **AgentGroupChat** orchestration.

Moderator Agent SK Plugin Functions (Tools)

DocumentIntelligencePlugin:

- `ExtractDocumentText(documentUrl)`: Calls Azure AI Document Intelligence, returns extracted text and field values
- `ValidateLicenseFields(extractedText, documentType)`: Validates required fields are present and legible
- `CheckExpiryDate(extractedText)`: Extracts and validates expiry dates against current date

ListingAnalysisPlugin:

- `CheckPriceAnomaly(listingPrice, categoryId, locationId)`: Compares submitted price against the distribution of similar listings; flags outliers below 40% or above 200% of category median
- `DetectSpamContent(descriptionText)`: Pattern-based + LLM detection of promotional spam, keyword stuffing, or copied descriptions
- `CheckDuplicateListing(titleText, providerId)`: Detects if a provider is submitting a near-duplicate of an existing listing (fuzzy match on title + same category)

ContentModerationPlugin:

- `FilterProhibitedContent(textContent)`: Checks for prohibited terms (weapons, illegal services, discriminatory language)
- `CheckImageContent(imageUrl)`: Calls Azure Content Moderator for explicit or misleading image content
- `ValidateContactInfoAbsence(textContent)`: Detects embedded phone numbers, emails, or external links in public-facing text fields

Moderator Agent Risk Scoring Protocol

For every new listing or product submission, the Moderator Agent produces a structured risk report with the following fields:

```
{
  "listing_id": "...",
  "overall_risk_level": "LOW" | "MEDIUM" | "HIGH",
  "overall_risk_score": 0-100,
  "risk_flags": [
    {
      "flag_type": "PRICE_ANOMALY" | "SPAM_CONTENT" | "DOCUMENT_UNREADABLE" |
        "DUPLICATE_LISTING" | "PROHIBITED_CONTENT" |
"MISSING_REQUIRED_DOCS" |
        "IMAGE_QUALITY_LOW" | "CONTACT_INFO_EMBEDDED",
      "severity": "WARNING" | "CRITICAL",
      "detail": "Human-readable explanation string",
      "auto_action": "NOTIFY_PROVIDER" | "BLOCK_SUBMISSION" | "FLAG_FOR_ADMIN"
    }
  ],
  "document_verification_results": [
    {
      "document_type": "...",
      "readable": true/false,
      "key_fields_present": true/false,
      "expiry_status": "VALID" | "EXPIRED" | "UNREADABLE",
      "confidence_score": 0-1
    }
  ],
  "agent_recommendation": "APPROVE" | "APPROVE_WITH_CONDITIONS" | "REJECT",
  "recommendation_reasoning": "Human-readable summary string"
}
```

Risk Score Interpretation:

- Score 0-30 (LOW): Agent recommends APPROVE. Admin receives a digest notification (not urgent).
- Score 31-60 (MEDIUM): Agent recommends APPROVE_WITH_CONDITIONS. Admin must review the specific flags before approving.
- Score 61-100 (HIGH): Agent recommends REJECT. Admin must review the full report; a rejection can be overridden but requires a written admin note.

Critical Auto-Block Rule: If the Moderator Agent detects `PROHIBITED_CONTENT` with CRITICAL severity, the listing is automatically placed in `BLOCKED` status regardless of overall risk score, and the admin is notified immediately via the highest-priority notification channel.

6.5 Multi-Agent Orchestration Protocol

The Pre-Booking Orchestration Scenario

This is the primary multi-agent use case in ShareGear: before a booking is finalized, both the Concierge Agent and the Moderator Agent must contribute information to produce a complete operational readiness assessment.

Orchestration Architecture: The ASP.NET Core API instantiates an `AgentGroupChat` containing both the Concierge Agent and the Moderator Agent. A `KernelFunctionTerminationStrategy` is configured to end the conversation when a structured completion signal is produced.

Orchestration Flow:

```

[Customer Submits Booking Request]
    |
    v
[BookingOrchestrationService instantiates AgentGroupChat]
    |
    v
[Concierge Agent – Turn 1: Availability & Geo Validation]
    → Calls GetProviderAvailability(providerId, requestedDate)
    → Calls ValidateLocationInServiceZone(customerLocation, providerId)
    → Produces: { availability_confirmed: bool, geo_valid: bool, conflict_details:
string | null }
    |
    v
[Moderator Agent – Turn 2: Provider Trust Assessment]
    → Reads current Moderator risk score for this provider (cached from last listing
review)
    → Checks provider's historical booking completion rate from SQL Server
    → Checks for any active admin flags on provider account
    → Produces: { provider_trust_score: 0-100, active_flags: [], trust_level:
"HIGH"|"MEDIUM"|"LOW" }
    |
    v
[AgentGroupChat Synthesis – Final Booking Decision Object]
    → {
        availability_confirmed: bool,
        geo_valid: bool,
        provider_trust_level: string,
        booking_recommendation: "PROCEED" | "PROCEED_WITH_WARNING" | "BLOCK",
        customer_warning_message: string | null,
        admin_alert_required: bool
    }
    |
    v
[BookingOrchestrationService processes the decision object]
    → If PROCEED: Booking created, provider notified
    → If PROCEED_WITH_WARNING: Booking created, customer shown non-blocking advisory
    → If BLOCK: Booking halted, customer shown reason, alternatives offered by
Concierge Agent

```

Provider Scheduling Conflict Resolution (Multi-Agent Sub-Flow)

When the Concierge Agent identifies an operator scheduling conflict (the provider's only registered operator is already assigned to another active booking at the same time):

1. The Concierge Agent signals the conflict to the AgentGroupChat.
 2. The Moderator Agent queries the provider's full operator roster.
 3. If an alternative operator exists on the provider's roster and is available: the Concierge Agent proposes the alternative operator to the customer ("Provider X can fulfill your request with a different operator, [Operator B], who has 5 years of experience with cranes and is available at your requested time").
 4. If no alternative operator is available: The AgentGroupChat produces a `CONFLICT_UNRESOLVABLE` signal. The Concierge Agent then immediately invokes `GetAlternativeServices()` to propose up to 3 other providers.
-

6.6 OCR Document Verification Flow

Trigger

Document verification is triggered in two scenarios:

1. A provider uploads documents during listing creation or profile update
2. A provider uploads operator license documents when registering a new operator

Flow

Step 1 — Document Upload: The provider uploads a document through the Angular UI. The file is sent to the `POST /api/providers/documents/upload` endpoint. The backend stores the file in secure blob storage (Azure Blob Storage) and creates a `DocumentVerificationJob` record in SQL Server with status `QUEUED`.

Step 2 — OCR Extraction: A background worker picks up the `QUEUED` job and invokes the Moderator Agent's `DocumentIntelligencePlugin.ExtractDocumentText()` function. Azure AI Document Intelligence processes the file and returns:

- Raw extracted text
- Structured field values (if the document type matches a pre-trained model — e.g., Egyptian national ID, driving license, business certificate)
- A confidence score for the extraction quality

Step 3 — Field Validation: The Moderator Agent validates the extracted fields according to the document type's expected schema:

- Equipment certificate: Equipment type, registration number, owner name, expiry date
- Operator license: Operator name, license class, issue date, expiry date

- Business registration: Company name, registration number, registered address

Step 4 — Expiry & Authenticity Checks:

- Expiry date is compared to the current date. Documents expiring within 30 days trigger a `WARNING` flag.
- Expired documents trigger a `CRITICAL` flag and cannot be used for listing approval.
- If key fields are missing or confidence score is below 0.7, the document is flagged as `UNREADABLE` and the provider is notified to re-upload a clearer scan.

Step 5 — Result Storage & Notification: The `DocumentVerificationJob` is updated with the full extraction results and risk assessment. The provider is notified of the verification outcome. If all documents for a listing pass verification, the listing's `PENDING_DOCUMENT_VERIFICATION` status clears and it advances to full `PENDING_REVIEW` for admin approval.

6.7 AI Review Filtering System

Purpose

Before any review is published on the platform, it passes through the Moderator Agent's content filter to detect:

- Spam or promotional content submitted as a review
- Personally identifiable information (phone numbers, addresses)
- Abusive, profane, or discriminatory language
- Content that violates platform review policies (e.g., reviewing something the user did not actually experience)

Flow

Step 1: User submits a review through the post-service or post-purchase review form.

Step 2: The review payload is sent to the Moderator Agent's `FilterProhibitedContent()` and `ValidateContactInfoAbsence()` functions.

Step 3: The LLM additionally scores the review for authenticity: Does the review content contextually match the type of service described? A review saying "great furniture store" on a crane service listing is anomalous and flagged for admin attention.

Step 4 — Decision:

- CLEAN: Review published immediately. Provider is notified.

- **FLAGGED_WARNING:** Review published but flagged in the admin queue for manual spot-check.
- **BLOCKED:** Review not published. User is informed the review violates policy and why. User can edit and resubmit once.

Step 5: Rating aggregates (provider's average rating) are updated in real time in SQL Server once a review is published.

6.8 AI Complaint Summarization

Purpose

When a customer or provider submits a complaint, the raw text (which may be lengthy, emotionally charged, and unstructured) is processed by the Moderator Agent to produce a concise, neutral, professional summary for the admin review queue.

Flow

Step 1: User submits complaint text (maximum 2,000 characters) and selects a complaint category (Service Quality / Operator Conduct / Payment Issue / Fraud / Safety Concern / Other).

Step 2: The Moderator Agent processes the complaint through a specialized summarization SK prompt function:

- **Input:** Raw complaint text + complaint category + booking/order reference data
- **Prompt Directive:** Summarize this complaint in 3-5 sentences. Be factual and neutral. Identify: the core issue, the claimed impact, and any specific evidence mentioned by the complainant. Do not add interpretation or take a position.
- **Output:** A structured summary object:

```
{ summary: string, identified_issue: string, claimed_impact: string, evidence_mentioned: string[], escalation_priority: "NORMAL" | "HIGH" | "URGENT" }
```

Step 3: The escalation priority is determined by keyword and semantic signals:

- **URGENT:** Keywords related to safety incidents, physical danger, fraud allegations with specific amounts, threats
- **HIGH:** Payment disputes, operator misconduct, significant service failure
- **NORMAL:** Quality complaints, communication issues, minor service deviations

Step 4: The admin receives the complaint summary card in their dashboard, ordered by escalation priority. The original raw text is always available by expanding the card — the AI summary is a productivity aid, not a replacement for the full complaint.

PART VII: Proactive Business Logic & Smart MVP Extensions

7.1 Automated Escrow Payment System

Design Rationale

The escrow system is the financial trust mechanism of the platform. It protects the customer (funds only release when service is confirmed complete) and motivates the provider (they know funds are secured for the job). For MVP, the escrow mechanism is implemented as a logical state machine in SQL Server, not as a separate banking integration — the payment gateway (e.g., Paymob or Stripe) captures the payment and holds it; the platform's escrow state machine controls the release signal.

Escrow State Machine

```
AWAITING_CAPTURE      (payment form opened, not yet paid)
  ↳ CAPTURED          (payment gateway confirms capture)
    ↳ HELD             (booking moves to ACTIVE status)
      ↳ RELEASED_TO_PROVIDER (completion confirmed)
      ↳ REFUNDED_TO_CUSTOMER (cancellation / dispute resolution
customer win)
    ↳ PARTIALLY_SETTLED   (dispute resolution – split decision)
```

Escrow Release Conditions

The escrow release API call is only made when ALL of the following are true:

1. Booking status is exactly `COMPLETED` (not `IN_PROGRESS`, not `PENDING_CUSTOMER_CONFIRMATION`)
2. The `CompletionConfirmedAt` timestamp is set (either by customer action or auto-confirmation timer)
3. No active dispute record exists linked to this booking
4. A minimum hold period of 30 minutes has elapsed since completion confirmation (fraud prevention cool-down)

Escrow Release Execution

1. The `EscrowReleaseService` is triggered as a background job (Hangfire or .NET IHostedService).
2. It validates all four conditions above in a database-level transaction.
3. If all conditions pass, it calls the payment gateway's "capture/transfer" API to move funds to the provider's registered payout account minus the platform commission.
4. Commission is held in the platform's settlement account.
5. A transaction record is written to SQL Server.
6. The provider receives a push notification and email with the payout details.

Commission Calculation Logic

```
Gross Amount = (Actual Service Hours × Hourly Rate) + Surcharges
Platform Commission = Gross Amount × Commission Rate (e.g., 10%)
Provider Net Payout = Gross Amount - Platform Commission
VAT Handling = Applied per Egyptian tax regulations on platform commission
```

Actual Hours Billing Rule: For MVP, the billing is based on the estimated hours agreed at booking. The provider can submit a time adjustment request (up to +2 hours above estimate) through the platform, which requires customer acknowledgment before escrow is modified. This prevents surprise bills.

Dispute-Triggered Escrow Freeze

When a dispute is opened, an `EscrowFreezeRecord` is created with:

- A timestamp
- A freeze reason
- The IDs of both parties

No automated system can release or refund frozen escrow. Only an admin action (via the Admin Dashboard's dispute resolution interface) can trigger the unfreezing and direct the funds.

7.2 Geo-Fencing & Machine Routing Intelligence

Service Zone Definition

Each service listing has a declared service zone defined by:

- A center point (provider's base location — governorate and district)
- A service radius (0–50 km, selectable in 5 km increments)

This is stored as a geospatial value in SQL Server (using the `geography` data type) for efficient proximity queries.

Geo-Validation at Booking

When a customer submits a booking request with a service address, the

`GeographyPlugin.ValidateLocationInServiceZone()` function:

1. Geocodes the customer's service address to coordinates (via Google Maps Geocoding API or Azure Maps)
2. Calculates the distance between the customer's location and the provider's service zone boundary
3. Returns: `InZone: bool`, `DistanceFromZoneBoundary: float (km)`

If In-Zone: Booking proceeds normally.

If Out-of-Zone (Within 10km of Zone Boundary):

The system offers the customer an out-of-zone service option with an automatically calculated travel surcharge. Formula: $\text{Distance Beyond Zone} \times \text{Provider's Declared Per-KM Rate}$. Customer must explicitly accept this surcharge to proceed.

If Out-of-Zone (More Than 10km Beyond Zone Boundary):

Booking is blocked for this provider. The Concierge Agent immediately suggests alternative providers whose zones cover the customer's requested location.

Proactive Geo-Alert for Providers

When a provider updates their operational address or service zone, the system automatically checks if any pending bookings from existing customers would become out-of-zone as a result of the change. Affected customers are notified and offered a cancellation with full refund.

7.3 Operator Conflict Resolution Engine

The Core Problem

A service provider may have multiple active service listings but a limited number of registered operators. Two bookings could theoretically be accepted for overlapping time slots if the same operator is the only one listed. The Operator Conflict Resolution Engine prevents this.

Conflict Detection Logic

The conflict engine runs at two points:

Point 1 — At Booking Request Submission: Before the booking request is stored, the system queries the provider's operator schedule:

```
SELECT * FROM OperatorAssignments
WHERE OperatorId IN (SELECT OperatorId FROM ServiceListingOperators WHERE
ListingId = @ListingId)
AND ScheduledStart < @RequestedEnd
AND ScheduledEnd > @RequestedStart
AND Status IN ('CONFIRMED', 'ACTIVE')
```

If this query returns any rows, a conflict exists.

Point 2 — At Provider Acceptance: A final conflict check is executed at the moment the provider taps "Accept." This double-check prevents the edge case where two bookings are simultaneously accepted before the first one is recorded.

Conflict Resolution Options

Option A — Alternative Operator Assignment: If the provider has other registered operators available at the requested time, the system presents them as assignment options at the acceptance step.

Option B — Negotiated Time Shift: The system suggests minor time adjustments (e.g., "This slot is unavailable. Could you offer the customer 2 hours later?") if that would resolve the conflict. The provider can send this alternative time offer to the customer through the platform.

Option C — Graceful Rejection with Alternatives: If no resolution is possible, the provider is guided to reject the conflicting booking with a pre-filled rejection reason, and the Concierge Agent immediately provides the customer with alternative providers.

Calendar Locking

Once a booking reaches **ACTIVE** status, the assigned operator's time slot is **hard-locked** in the calendar. The provider cannot manually override this lock from their dashboard — only an admin can force-unlock in exceptional circumstances (documented in audit log).

The Unresponsive Provider Problem

A provider that accepts bookings but then fails to respond to incoming requests is one of the most damaging experience failures on a service platform. ShareGear addresses this with a tiered escalation system.

Escalation Timeline (2-Hour Provider Response Window)

Elapsed Time	Action
T+0:00	Booking submitted. Provider notified via SignalR, push, and SMS.
T+0:30	First reminder: Push notification to provider.
T+1:00	Second reminder: SMS to provider (more urgent tone).
T+1:30	Third reminder: Push + SMS + in-app dashboard alert with "Booking at risk of expiry" warning.
T+2:00	SLA Breached. Booking enters <code>PROVIDER_UNRESPONSIVE</code> state. Fallback flow initiates.

Fallback Flow

Step 1 — Customer Notification: Customer is immediately notified: "The provider has not responded to your request. We're finding you an alternative."

Step 2 — AI Alternative Discovery: The Concierge Agent is invoked to find the top 3 semantically similar services with:

- Availability confirmed for the customer's requested date/time
- Provider response rate above 80%
- Geo-zone match
- Ordered by: relevance score × rating score

Step 3 — Backup Provider Offer: Customer is presented with the alternatives within 2-5 minutes and can accept one with a single tap (their booking details and payment method are pre-filled).

Step 4 — Original Provider Penalty:

The unresponsive provider's account receives an automated `UNRESPONSIVE_INCIDENT` log entry.

After 3 such incidents within 30 days:

- Provider's search ranking is demoted (a `ReputationPenaltyScore` flag is applied)
- Provider receives an in-app warning with a link to their performance dashboard
- After 5 incidents: Provider account is flagged for admin review and potential suspension

Step 5 — Holdout Option: If the customer prefers to wait for the original provider rather than switch, they can choose to extend the response window by 2 additional hours. This is a one-time option per booking.

7.5 Dynamic Pricing Signals Engine

Purpose

While providers set their own base rates, the platform provides pricing intelligence signals to help providers stay competitive and to inform customers whether a price is fair. This is not a forced pricing system — it is an advisory layer.

Provider-Facing Pricing Intelligence (During Listing Creation)

When a provider enters a price during listing creation, the backend queries the SQL Server aggregate of active listings in the same category and geographic region and returns:

```
{
  "category_median_price": 850,           // EGP/hour
  "category_25th_percentile": 600,
  "category_75th_percentile": 1100,
  "your_price_percentile": 35,
  "market_positioning": "BELOW_MARKET", // or AT_MARKET, ABOVE_MARKET, OUTLIER
  "recommendation": "Your price is competitive. Listings priced within 10-15%
above median receive 22% more booking requests on average."
}
```

This signal is displayed as a non-blocking advisory widget in the listing creation form.

Customer-Facing Price Fairness Indicator

On each service listing detail page, a subtle badge indicates:

- "Great Value" (price in bottom 25th percentile of category)
- "Market Rate" (price between 25th and 75th percentile)
- "Premium" (price in top 25th percentile)

This is calculated at listing display time, not stored as a static field, so it updates automatically as market pricing evolves.

Surge Pricing Advisory (Non-Automated)

During periods of high demand (detected by the admin analytics dashboard — e.g., a surge in bookings for a specific equipment type during a construction season), the Admin can activate a **Surge Advisory** for a category. This does not change any listing prices. It displays a banner to customers: "High demand for [crane services] in your area. Booking in advance is recommended." This is a transparency feature, not price manipulation.

7.6 Trust & Reputation Scoring System

Provider Trust Score

Every provider has a system-computed Trust Score (0-100) that is used internally for search ranking, booking fast-tracking, and risk assessment. It is not directly visible to customers but powers the "Verified" and "Top Provider" badges.

Trust Score Components:

Component	Weight	Source
Average customer rating	30%	Reviews table
Booking completion rate	25%	Completed / Total Accepted bookings
Provider response rate	20%	Responses within SLA / Total requests
Document verification completeness	15%	Verified docs / Required docs
Unresponsive incident count (penalty)	-10%	IncidentLog table

Trust Level Bands:

- 80-100: "Top Provider" — eligible for featured placement
- 60-79: "Verified Provider" — standard search placement
- 40-59: "Standard" — no special placement, review reminder sent monthly
- Below 40: "At Risk" — admin notified, account review initiated

Customer Trust Score

Customers also have an internal trust score (not shown to providers) used to manage platform risk:

- Booking cancellation rate (after escrow capture, penalizes high rates)
- Dispute frequency (frequent disputers are flagged for review)
- Review authenticity score (from Moderator Agent's review analysis)
- Payment failure rate (incomplete payments)

Customers below a threshold are required to verify additional identity before booking premium (high-value) services.

7.7 SLA Enforcement & Dispute Resolution Engine

Service Level Agreements Enforced by the Platform

SLA	Threshold	Consequence
Provider response time	2 hours	Fallback flow triggers (Section 7.4)
Customer completion confirmation	4 hours	Auto-confirmation triggers
Customer marketplace confirmation	72 hours	Auto-confirmation triggers
Admin dispute resolution	3 business days	Escalation to senior admin
Listing review (by AI)	60 seconds	Alert to DevOps if exceeded
Listing review (by Admin)	24 hours	Escalation notification

Dispute Resolution Workflow

- Step 1 — Dispute Submission:** Customer submits a dispute with: category, description, and optional evidence uploads (photos, screenshots).
- Step 2 — Moderator Agent Analysis:** The Moderator Agent analyzes the dispute text against the booking record: timeline of events, chat messages, operator logs, and any provider-submitted issue flags. It produces a dispute summary and a preliminary recommendation.
- Step 3 — Provider Right of Reply:** The provider has 24 hours to submit their perspective. The Moderator Agent summarizes the provider response as well.

Step 4 — Admin Decision: Admin reviews both summaries (and originals), the dispute history of both parties, and makes a resolution decision:

- **Full refund to customer:** Escrow fully returned
- **Full release to provider:** Escrow released to provider
- **Partial settlement:** Escrow split (admin defines the percentage)
- **Service re-do:** Rare — provider agrees to redo the service without additional charge

Step 5 — Communication & Closure: Both parties are notified of the decision with a brief explanation. The dispute record is closed and contributes to both parties' trust score calculations.

7.8 Smart Cancellation Policy Engine

Provider-Initiated Cancellation

If a provider cancels after accepting a booking:

- Less than 48 hours before service: Full customer refund + provider receives a cancellation penalty recorded in their incident log.
- More than 48 hours before service: Full customer refund, no penalty (within 1 cancellation per 30 days).
- More than 3 cancellations in 30 days: Account flagged for admin review.

Customer-Initiated Cancellation

Cancellation fees are based on proximity to the service start time:

- Cancelled more than 48 hours before: Full refund
- Cancelled 24-48 hours before: 75% refund (25% non-refundable processing fee)
- Cancelled 4-24 hours before: 50% refund
- Cancelled less than 4 hours before: No refund (except in documented emergency — admin discretion)

Emergency Override: A customer can flag a cancellation as an emergency (medical, force majeure). This suspends the fee pending admin review. Admin has 24 hours to approve or reject the override claim.

Marketplace Cancellation

For marketplace purchases:

- Cancelled before seller confirms dispatch: Full refund, no fee
 - Cancelled after dispatch but before delivery: Buyer pays return shipping cost (if applicable); remainder refunded
 - No cancellation after confirmed collection/delivery (dispute process must be used instead)
-

7.9 New Provider Onboarding Risk Pipeline

The Problem

Fraudulent providers are a significant risk on any marketplace. A new provider account with a freshly uploaded listing should not receive the same trust as a long-standing verified provider.

Tiered Onboarding Gates

Tier 0 — New Unverified Provider:

- Can create listings but listings are not published until document verification passes
- Cannot receive bookings
- Limited to 3 listing submissions pending review (prevents spam)

Tier 1 — Document Verified (Moderator Agent passed + Admin approved at least 1 listing):

- Up to 5 active listings
- Maximum booking value: 5,000 EGP per booking
- Payment released after 7-day hold post-completion (additional fraud protection)

Tier 2 — Established Provider (5+ completed bookings, rating $\geq$ 4.0, no active flags):

- Unlimited active listings
- No booking value cap
- Standard payment release (no additional hold)

Tier 3 — Top Provider (Trust Score $\geq$ 80, 20+ completions, rating $\geq$ 4.5):

- Featured placement eligibility
 - Priority booking notifications
 - Dedicated provider success contact (admin)
-

7.10 Smart Listing Quality Improvement Suggestions

Rationale

Providers on industrial platforms are often operators, not marketers. Their listing descriptions may be technically accurate but poorly written, missing key information, or not optimized for search discovery. The platform provides proactive quality improvement suggestions after a listing is created.

Quality Score Calculation

After a listing is created, a background process computes a Listing Quality Score (0-100):

Factor	Max Points
Description length and completeness	20
Photo count (max score at 6+)	15
Structured specifications filled	15
Pricing detail completeness	10
Operator profile completeness	15
Document verification completeness	15
Service zone definition specificity	10

AI-Generated Improvement Recommendations

For listings with Quality Score below 70, the Concierge Agent (in provider-advisory mode) generates a personalized improvement checklist:

- "Your listing description is under 150 characters. Adding more detail about the specific tasks you can handle can increase booking inquiries by up to 40%."
- "You have 2 photos. Listings with 6+ photos receive significantly more customer interest."
- "Your operator's license documents are not yet verified. Adding them will display a 'Verified Operator' badge and increase customer trust."

These suggestions appear in the provider's listing management dashboard as a non-intrusive "Improve Your Listing" card.

8.1 Admin Command Center

The Admin Dashboard is a dedicated Angular module (route-guarded with the `Admin` role) that serves as the central control plane for all platform operations.

Primary Dashboard View — Live Operations

The admin's default view presents a real-time operational summary:

- **Active Bookings Counter:** Total bookings currently `IN_PROGRESS`
- **Pending Provider Responses:** Bookings in `PENDING_PROVIDER_RESPONSE` status, ordered by time elapsed
- **SLA Breach Alerts:** Any booking approaching or past its provider response SLA
- **Listings Pending Review:** Queue count with oldest-first ordering
- **Open Disputes:** Count and escalation priority breakdown
- **Moderator Agent Activity Feed:** Real-time log of AI processing events (last 50 events)

Listing Review Queue

The listing review queue presents each pending listing with:

- AI-generated risk score and risk level badge (LOW/MEDIUM/HIGH)
- Full Moderator Agent risk report (expandable)
- Document verification results for each uploaded document
- Side-by-side comparison with similar approved listings (for price context)
- One-click Approve / Reject / Request More Info actions
- For rejections: Mandatory rejection reason selection + optional admin note (sent to provider)

Efficiency Design: Listings with AI risk score ≤ 25 (clearly clean) have an "Approve All Clean Listings" batch action available, allowing admins to process multiple low-risk listings in a single confirmation.

User Management

- Search users by name, email, phone, or role
- View user activity timeline (registrations, bookings, purchases, complaints)
- Trust score history (trend chart)
- Actions: Verify, Suspend, Unsuspend, Change Role, Delete Account

- All actions logged to immutable AuditLog table with admin ID, timestamp, and reason
-

8.2 AI Risk Dashboard

Moderator Agent Performance Metrics

- Total items reviewed today / this week / this month
- Approval rate breakdown (% approved without admin review, % requiring admin review, % rejected by AI)
- False positive rate (items the AI flagged HIGH but admin approved — tracked for model feedback)
- Average processing time per item

Risk Trend Analysis

- Chart: Daily distribution of risk scores across listing submissions (trend over 30 days)
- Category-level risk breakdown: Which service or product categories are generating the most high-risk flags?
- Provider-level risk: A ranked list of providers ordered by historical risk incident frequency

AI Configuration Panel

Admin can adjust operational parameters without a code deployment:

- Similarity threshold for semantic search (default: 0.65)
 - Provider response SLA duration (default: 2 hours)
 - Auto-confirmation window for customers (default: 4 hours)
 - Price anomaly detection thresholds (low outlier %: default 40%, high outlier %: default 200%)
 - Risk score bands (boundary between LOW/MEDIUM/HIGH)
 - Content moderation keyword blacklist management
-

8.3 Analytics & Reporting

Service Analytics

- Most requested service categories (ranked by booking count)
- Booking conversion rate: Search query → Listing view → Booking started → Booking completed

- Provider performance distribution: Rating histogram
- Geographic heat map: Where are bookings being requested? (Governorate-level)
- Average booking value by category
- Revenue by time period (daily, weekly, monthly)

Marketplace Analytics

- Active listings count by category
- Average time from listing published to first inquiry
- Purchase conversion rate: Listing view → Purchase
- Average sale price by category
- Seller activity metrics

AI Search Analytics

- Total search queries processed (daily)
 - Semantic vs. keyword pattern distribution (for internal optimization)
 - Queries with zero relevant results (identifies gaps in supply)
 - Most common extracted intent categories (tells platform which service types to recruit)
 - Concierge Agent alternative suggestion acceptance rate (when a backup is offered, how often do users take it?)
-

PART IX: Security & Compliance Framework

9.1 Authentication Security

- All passwords are hashed using BCrypt with a cost factor of 12
- API keys for third-party integrations (AI, payment gateway, maps) are stored in environment variables or Azure Key Vault — never in code or database
- JWT tokens are signed with RS256 (asymmetric keys) for enhanced security
- Rate limiting is applied at the API gateway level: 100 requests/minute per IP for unauthenticated routes, 500 requests/minute for authenticated routes
- All authentication events (login, logout, token refresh, failed attempts) are logged to the SecurityAuditLog table

9.2 Data Privacy

- Customer phone numbers are masked in any provider-visible context (displayed as partial:

"010-XXXX-1234")

- Operator personal data (full legal name, national ID) is stored in an encrypted column using AES-256
- Users can request account deletion; associated personal data is anonymized while transactional records (bookings, payments) are retained for legal/tax compliance periods
- Chat message content is stored encrypted at rest

9.3 Payment Security

- The ASP.NET Core backend never stores raw payment card data — only tokenized references from the payment gateway (PCI-DSS compliance boundary)
- All payment API calls are made server-side only — the Angular frontend communicates with the payment gateway's hosted tokenization form, not its raw API
- Escrow state transitions are wrapped in database transactions with optimistic concurrency control to prevent double-release race conditions
- Payment webhook callbacks from the gateway are validated using signature verification before being processed

9.4 AI Safety Guardrails

- All LLM prompt templates are version-controlled and subject to team review before deployment
- LLM responses are post-processed through a length and content sanity check before being returned to any user-facing interface
- The Moderator Agent never makes autonomous decisions on user accounts — it only produces recommendations; all account actions require admin confirmation
- AI processing logs (inputs, outputs, latency) are stored for debugging and audit purposes, with a 90-day retention policy

PART X: MVP Implementation Roadmap

Phase 1 — Core Infrastructure & Authentication (Weeks 1-3)

- ASP.NET Core project setup with Identity, JWT, and RBAC
- SQL Server schema implementation (Users, Roles, Listings, Bookings, Products, Orders, Reviews, Complaints, AuditLog)
- Angular project structure, routing, and authentication module
- Basic provider and customer registration/login flows with phone OTP

- Admin role and basic dashboard shell

Phase 2 — Service Listing & Marketplace Listing (Weeks 4-6)

- Provider listing creation multi-step form (Angular)
- Service listing API endpoints (CRUD)
- Marketplace product listing API endpoints (CRUD)
- Azure Blob Storage integration for photo and document uploads
- Basic listing display and detail pages
- Category and filter browsing

Phase 3 — AI Search & Moderator Agent (Weeks 7-9)

- Semantic Kernel integration in ASP.NET Core
- Embedding generation pipeline and Qdrant setup
- Semantic search endpoint and Angular search bar integration
- Moderator Agent implementation with OCR, price anomaly, and content checks
- Admin listing review queue with AI risk reports

Phase 4 — Booking System & Escrow (Weeks 10-12)

- Complete booking request flow (Angular multi-step form)
- Booking state machine implementation
- Provider acceptance/rejection flow
- Payment gateway integration with escrow logic
- Automated escrow release background service
- Booking tracker views for customer and provider

Phase 5 — SignalR, Notifications & Operator Flows (Weeks 13-14)

- SignalR hub implementation
- Real-time chat system
- Push notification and SMS integration
- Notification event catalog implementation
- Operator scheduling and conflict detection engine
- Backup notification and fallback flow

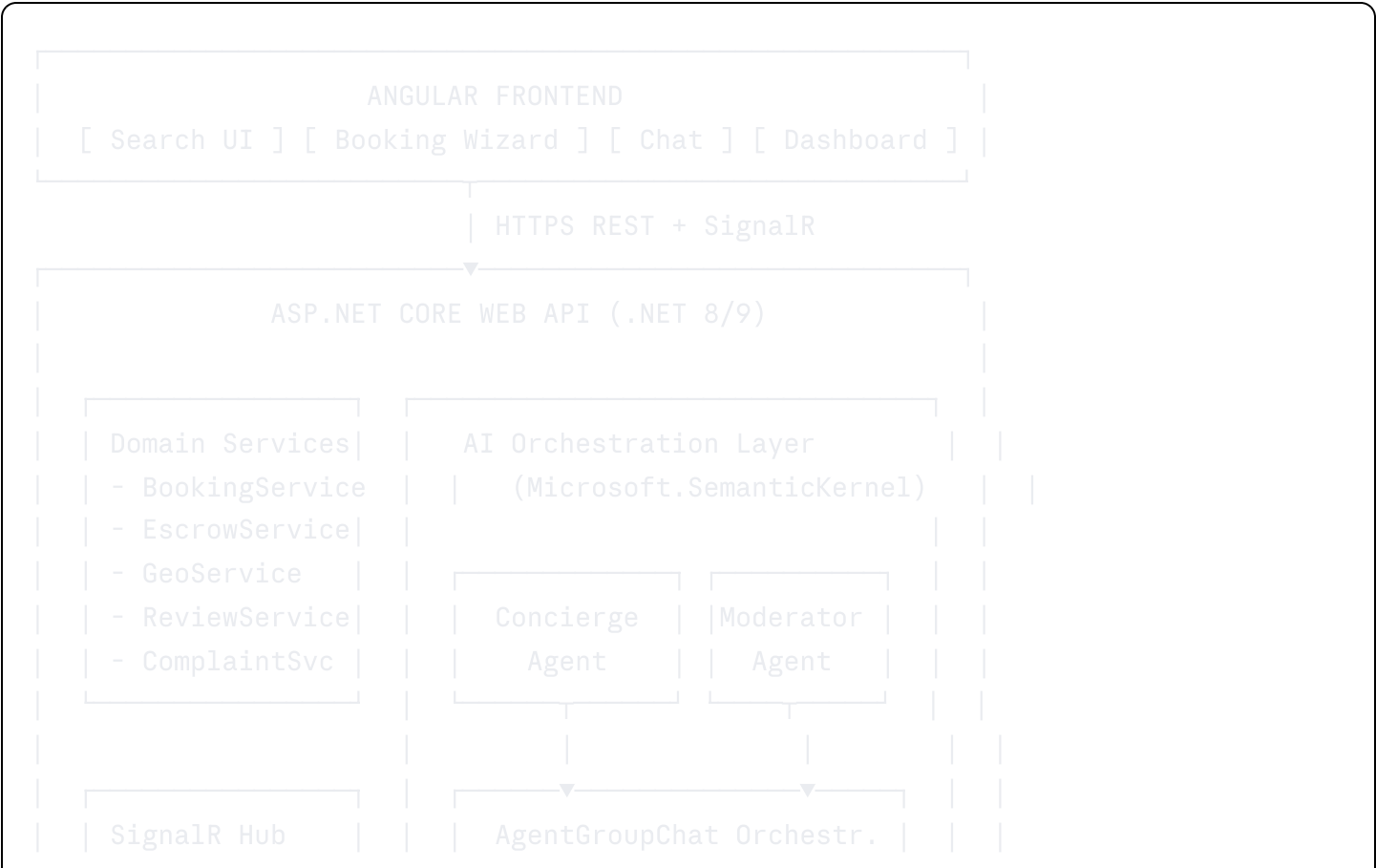
Phase 6 — Multi-Agent Orchestration & Advanced AI (Weeks 15-16)

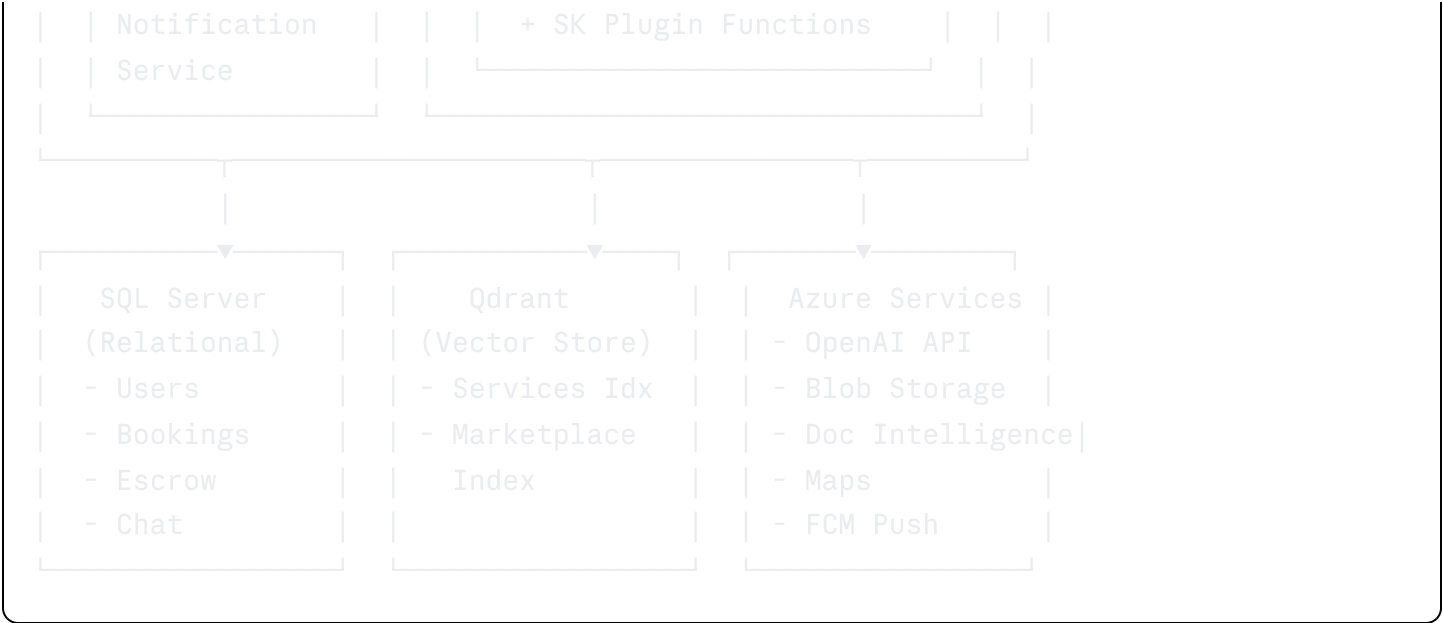
- Concierge Agent full implementation with SK plugin functions
- Pre-booking AgentGroupChat orchestration
- AI review filtering
- AI complaint summarization
- Listing quality score and improvement suggestions

Phase 7 — Analytics, Refinement & QA (Weeks 17-18)

- Admin analytics dashboard
- Trust and reputation scoring system
- Dynamic pricing signals engine
- Dynamic provider tier management
- End-to-end integration testing
- Performance optimization (caching, query optimization, Qdrant index tuning)
- Security review and penetration testing checklist

Final Architecture Summary





End of ShareGear MVP Functional Specification — Version 1.0
Prepared for Team 5 Graduation Project